

C 언어 프로그램 설명 및 결과

1. 프로그램 1.1 - cal_scores.c

📖 설명

- 최대 100개의 점수를 저장할 수 있는 배열 `scores`를 선언
- `get_max_score(int n)` 함수는 `n`개의 점수 중 **최대값**을 찾아 반환

🔍 코드

```
#include <stdio.h>
#define MAX_ELEMENTS 100

int scores[MAX_ELEMENTS]; // 자료구조

int get_max_score(int n){
    int i, largest;
    largest = scores[0];

    for (i = 0; i < n; i++){
        if(scores[i] > largest){
            largest = scores[i];
        }
    }
    return largest;
}
```

🔗 실행 결과

(예를 들어 `scores = {10, 25, 3, 42, 8}`일 때)

최대 점수: 42

2. 프로그램 1.2 - cal_time.c

📖 설명

- 프로그램 실행 시간을 측정하는 코드
- `clock_t`를 사용하여 루프 실행 시간을 계산

🔍 코드

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(){
    clock_t start, stop;
    double duration;

    start = clock();

    for (int i = 0; i < 1000000; i++){
        printf("%d\n", i);
    }

    stop = clock();
    duration = (double)(stop - start) / CLOCKS_PER_SEC;

    printf("수행시간은 %f초입니다.\n", duration);
    return 0;
}
```

🔗 실행 결과

```
0
1
2
...
999999
수행시간은 2.345678초입니다.
```

(실행 환경에 따라 달라질 수 있음)

3. 프로그램 2.1 - factorial.c

📖 설명

- 재귀 함수를 이용하여 팩토리얼을 계산

🔍 코드

```
#include <stdio.h>

int factorial(int n){
    if(n <= 1) return 1;
    else return n * factorial(n-1);
}
```

🔗 실행 결과

(입력: `factorial(5)`)

120

4. 프로그램 2.2 - `print_fac.c`

📖 설명

- 재귀 과정이 출력되는 팩토리얼 계산 프로그램

🔍 코드

```
#include <stdio.h>

int factorial(int n){
    printf("factorial(%d)\n", n);
    if(n <= 1) return 1;
    else return n * factorial(n-1);
}
```

🔗 실행 결과

(입력: `factorial(5)`)

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
120
```

5. 프로그램 2.3 - `cal_factorial.c`

📖 설명

- 반복문을 이용한 팩토리얼 계산 프로그램

🔍 코드

```
#include <stdio.h>
```

```
int factorial_iter(int n){
    int i, result = 1;
    for (i = 1; i <= n; i++) result *= i;
    return result;
}
```

실행 결과

(입력: `factorial_iter(5)`)

120

6. 프로그램 2.4 - `cal_square.c`

설명

- 반복문을 이용한 거듭제곱 계산 프로그램

코드

```
#include <stdio.h>

double slow_power(double x, int n){
    int i;
    double result = 1.0;

    for (i = 0; i < n; i++){
        result *= x;
    }
    return result;
}
```

실행 결과

(입력: `slow_power(2, 5)`)

32.000000

7. 프로그램 2.5 - `flow_square.c`

설명

- 재귀 함수를 이용한 거듭제곱 계산 프로그램 (빠른 거듭제곱 알고리즘 사용)

 코드

```
#include <stdio.h>

double power(double x, int n){
    if(n == 0) return 1;
    else if (n % 2 == 0) return power(x * x, n / 2);
    else return x * power(x * x, (n - 1) / 2);
}
```

 실행 결과

(입력: power(2, 5))

32.000000

8. 프로그램 2.8 - hanoi.c

 설명

- 하노이의 탑 문제를 재귀적으로 해결

 코드

```
#include <stdio.h>

void hanoi_tower(int n, char from, char tmp, char to){
    if (n == 1) printf("원판 1을 %c에서 %c으로 옮긴다.\n", from, to);
    else {
        hanoi_tower(n-1, from, to, tmp);
        printf("원판 %d을 %c에서 %c으로 옮긴다.\n", n, from, to);
        hanoi_tower(n-1, tmp, from, to);
    }
}

int main(){
    hanoi_tower(4, 'A', 'B', 'C');
    return 0;
}
```

 실행 결과

원판 1을 A에서 B으로 옮긴다.
원판 2을 A에서 C으로 옮긴다.

...

